Confidential

hexens × FUEL

MAY.24

SECURITY REVIEW REPORT FOR FUEL

Confidential

LIMITATIONS ON DISCLOSURE AND USAGE OF THIS REPORT

This report has been developed by the company Hexens (the Service Provider) based on the Security Review of Fuel (the Client). The document contains vulnerability information and remediation advice.

The information, presented in this report is confidential and privileged. If you are reading this report, you agree to keep it confidential, not to copy, disclose or disseminate without the agreement of Fuel.

If you are not the intended recipient of this document, remember that any disclosure, copying or dissemination of it is forbidden.

Confidential

CONTENTS

Confidential

ABOUT HEXENS

Hexens is a cybersecurity company that strives to elevate the standards of security in Web 3.0, create a safer environment for users, and ensure mass Web 3.0 adoption.

Hexens has multiple top-notch auditing teams specialized in different fields of information security, showing extreme performance in the most challenging and technically complex tasks, including but not limited to: Infrastructure Audits, Zero Knowledge Proofs / Novel Cryptography, DeFi and NFTs. Hexens not only uses widely known methodologies and flows, but focuses on discovering and introducing new ones on a day-to-day basis.

In 2022, our team announced the closure of a $4.2 million seed round led by IOSG Ventures, the leading Web 3.0 venture capital. Other investors include Delta Blockchain Fund, Chapter One, Hash Capital, ImToken Ventures, Tenzor Capital, and angels from Polygon and other blockchain projects.

Since Hexens was founded in 2021, it has had an impressive track record and recognition in the industry: Mudit Gupta - CISO of Polygon Technology - the biggest EVM Ecosystem, joined the company advisory board after completing just a single cooperation iteration. Polygon Technology, 1inch, Lido, Hats Finance, Quickswap, Layerswap, 4K, RociFi, as well as dozens of DeFi protocols and bridges, have already become our customers and taken proactive measures towards protecting their assets.

SCOPE

The analyzed resources are located on:

https://github.com/FuelLabs/fuel-rollup/tree/0f2f6dfc4c6919a359ac7b5e004c803479562d8e

Confidential

AUDITING DETAILS

STARTED
21.05.2024

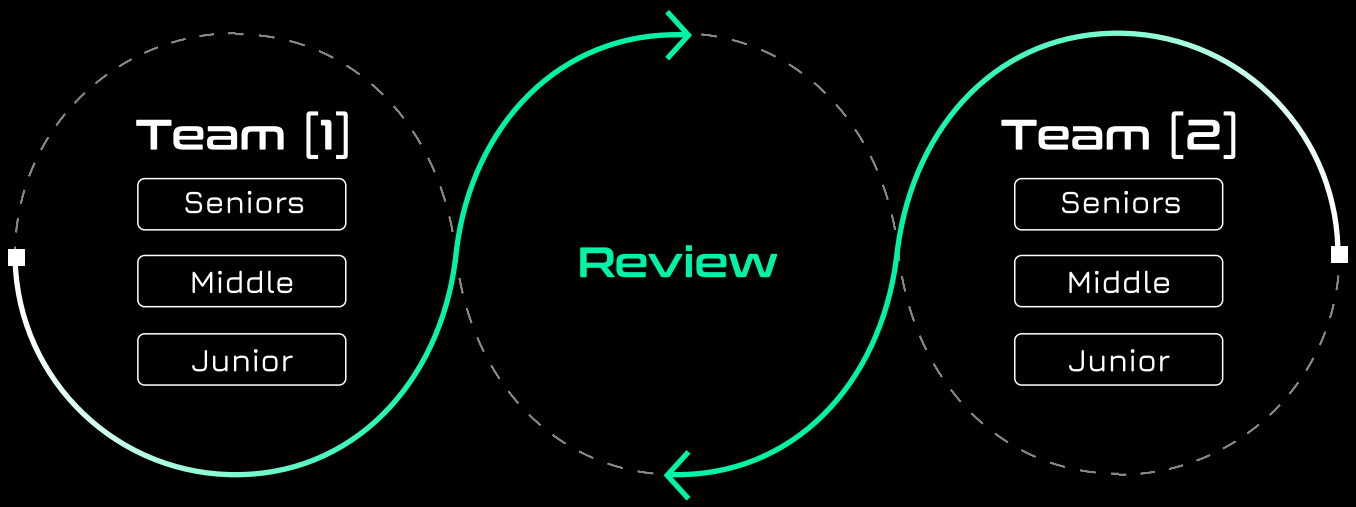
DELIVERED
27.05.2024

Review Led by

MIKHAIL EGOROV
Senior Security Researcher | Hexens

HEXENS METHODOLOGY

Hexens methodology involves 2 teams, including multiple auditors of different seniority, with at least 5 security engineers. This unique cross-checking mechanism helps us provide the best quality in the market.



SEVERITY STRUCTURE

The vulnerability severity is calculated based on two components

- Impact of the vulnerability
- Probability of the vulnerability

Impact	Probability			
	rare	unlikely	likely	very likely
Low/Info	Low/Info	Low/Info	Medium	Medium
Medium	Low/Info	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

SEVERITY CHARACTERISTICS

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical

Vulnerabilities with this level of severity can result in significant financial losses or reputational damage. They often allow an attacker to gain complete control of a contract, directly steal or freeze funds from the contract or users, or permanently block the functionality of a protocol. Examples include infinite mints and governance manipulation.

Confidential

High

Vulnerabilities with this level of severity can result in some financial losses or reputational damage. They often allow an attacker to directly steal yield from the contract or users, or temporarily freeze funds. Examples include inadequate access control integer overflow/underflow, or logic bugs.

Medium

Vulnerabilities with this level of severity can result in some damage to the protocol or users, without profit for the attacker. They often allow an attacker to exploit a contract to cause harm, but the impact may be limited, such as temporarily blocking the functionality of the protocol. Examples include uninitialized storage pointers and failure to check external calls.

Low

Vulnerabilities with this level of severity may not result in financial losses or significant harm. They may, however, impact the usability or reliability of a contract. Examples include slippage and front-running, or minor logic bugs.

Informational

Vulnerabilities with this level of severity are regarding gas optimizations and code style. They often involve issues with documentation, incorrect usage of EIP standards, best practices for saving gas, or the overall design of a contract. Examples include not conforming to ERC20, or disagreement between documentation and code.

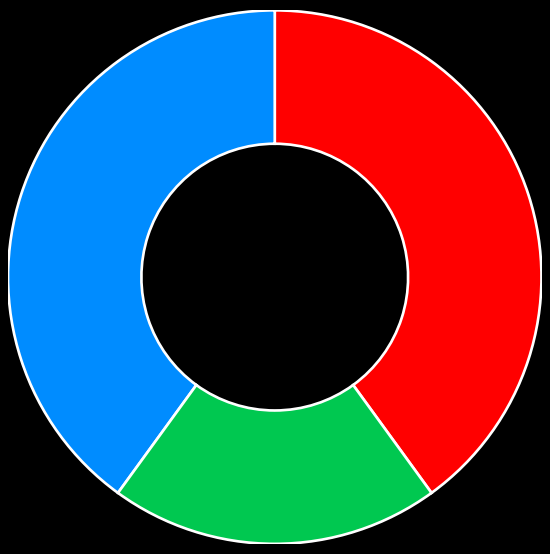
ISSUE SYMBOLIC CODES

Every issue being identified and validated has its unique symbolic code assigned to the issue at the security research stage. Cause of the vulnerability reporting flow design, some of the rejected issues could be missing.

FINDINGS SUMMARY

Severity	Number of Findings
Critical	0
High	2
Medium	0
Low	1
Informational	2

Total: 5



- High
- Low
- Informational

WEAKNESSES

This section contains the list of discovered weaknesses.

FUEL3-6

ENCODEBURN INCORRECTLY ENCODES COIN MESSAGE

SEVERITY: High

PATH:

fuel-rollup/contracts/utils/ProtoEncoding.sol:L110

REMEDIATION:

Don't add uint8(1) as the length when encoding the coin message in the encodeBurn() function.

STATUS:

DESCRIPTION:

The **encodeBurn()** function is not correctly encoding the coin message in Protobuf format. This issue can cause problems when decoding the coin message, including **denom** and **amount** fields, following the **Protobuf** specification. The issue arises since the current implementation mistakenly includes a length value of **uint8(1)**.

Consequently, making a withdrawal by calling **withdraw()** of **SequencerInterface** will not be possible.

In the **encodeBurn()** function, the second field is being encoded with the **LEN** wire type (0x12), but it incorrectly includes a field length value of **uint8(1)**.

Additionally, **encode_packed_repeated()** adds another length value of **encode_varint(uint64(b.length))**. Refer to line 178 of ProtoEncoding.sol.

As a result, the decoder will use length of 1 and either return the length of **encodeCoin** instead of the actual **encodeCoin** data, or encounter a panic when decoding a varint if the length of **encodeCoin** exceeds 256.

For further information on Protobuf encoding visit Encoding.

```
function encodeBurn(
    address from,
    string memory denom,
    uint256 amount
) public pure returns (bytes memory) {
    bytes memory coin = encodeCoin(denom, amount);

    return
        abi.encodePacked(
            bytes1(0x0a), // first field, length deliminated
            uint8(42), // length
            from.toHexString(), // from address
            bytes1(0x12), // second field, length deliminated
            uint8(1),
            encode_packed_repeated(coin)
        );
}
```

FUEL3-8

PROTOENCODING LACKS IMPLEMENTATION FOR FUNCTIONS

SEVERITY: High

PATH:

fuel-rollup/contracts/utils/ProtoEncoding.sol:L148-L158

REMEDIATION:

Implement encodeRedelegate(), encodeClaimRewards(), and encodeUnbond() of the ProtoEncoding contract.

STATUS:

DESCRIPTION:

The **encodeRedelegate()**, **encodeClaimRewards()**, and **encodeUnbond()** functions of the **ProtoEncoding** contract do not have an implementation yet and currently revert.

```
function encodeRedelegate() public pure returns (bytes memory) {
    revert("TODO");
}

function encodeClaimRewards() public pure returns (bytes memory) {
    revert("TODO");
}

function encodeUnbond() public pure returns (bytes memory) {
    revert("TODO");
}
```

As a result, the functions **unbond()**, **claimRewards()**, and **redelegate()** of the **SequencerInterface** contract will revert when called.

FUEL3-7

PROTOBUF ENCODING LIMITS LENGTH TO 1-BYTE VARINT

SEVERITY: Low

PATH:

fuel-rollup/contracts/utils/ProtoEncoding.sol:L46

REMEDIATION:

See description.

STATUS:

DESCRIPTION:

Functions in the **ProtoEncoding** contract, such as **encodeSend()**, assume that the length is represented as a 1-byte varint.

For instance, on line 46, if **coin.length** exceeds 255, using **uint8(coin.length)** will truncate the length value, leading to an incorrect Protobuf encoding.

To address this limitation, it's recommended to utilize higher-level Protobuf encoding methods such as **encode_string()**, **encode_bytes()**, and **encode_length_delimited()** from **ProtobufLib**. These functions handle encoding of a generic varint and guarantee the accurate length encoding for values exceeding 255.

For further information on Protobuf encoding visit Encoding.

```
function encodeSend(
    address sender,
    address to,
    string memory denom,
    uint256 amount
) public pure returns (bytes memory) {
    bytes memory coin = encodeCoin(denom, amount);

    return
        abi.encodePacked(
            bytes1(0x0a), // first field, length deliminated
            uint8(42), // length
            sender.toHexString(), // sender
            bytes1(0x12), // second field, length deliminated
            uint8(42), // length
            to.toHexString(), // to_address
            bytes1(0x1a), // third field, length deliminated
            uint8(coin.length),
            coin
        );
}
```

Use ProtobufLib to perform Protobuf encoding as follows:

```
function encodeSend(
    address sender,
    address to,
    string memory denom,
    uint256 amount
) public pure returns (bytes memory) {
    bytes memory coin = encodeCoin(denom, amount);

    return
        abi.encodePacked(
            bytes1(0x0a), // first field, length deliminated
            encode_bytes(sender.toHexString()),
            bytes1(0x12), // second field, length deliminated
            encode_bytes(to.toHexString()),
            bytes1(0x1a), // third field, length deliminated
            encode_bytes(coin)
        );
}
```

FUEL3-9

SEQUENCERPROXY SHOULD CALL _DISABLEINITIALIZERS IN CONSTRUCTOR

SEVERITY: Informational

PATH:

fuel-rollup/contracts/sequencer/SequencerProxy.sol

REMEDIATION:

Include a call to _disableInitializers() in the constructor of SequencerProxy.

STATUS:

DESCRIPTION:

SequencerProxy is an upgradeable contract and should invoke **_disableInitializers()** in the constructor to avoid the unintended initialization of the implementation contract.

```
// SPDX-License-Identifier: Apache 2.0
pragma solidity ^0.8.20;

import "@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol";
import "@openzeppelin/contracts-upgradeable/access/
AccessControlUpgradeable.sol";

import { ISequencerProxy } from "../interfaces/ISequencerProxy.sol";
import { Pausable } from "../utils/Pausable.sol";

contract SequencerProxy is
 ISequencerProxy,
 Initializable,
 AccessControlUpgradeable,
 Pausable
{
 bytes32 public constant INTERFACE_ROLE = keccak256("INTERFACE_ROLE");

 function initialize() external initializer {
  _grantRole(DEFAULT_ADMIN_ROLE, _msgSender());
  _grantRole(PAUSER_ROLE, _msgSender());
  __Pausable_init();
 }

 /// @inheritdoc ISequencerProxy
 function deposit(
  address depositor,
  address recipient,
  uint256 amount,
  uint256 lockup
 ) external whenNotPaused onlyRole(INTERFACE_ROLE) {
  emit Deposit(depositor, recipient, amount, lockup);
 }
```

```
  /// @inheritdoc ISequencerProxy
  function authorize(
   address sender,
   bytes calldata data
  ) external whenNotPaused onlyRole(INTERFACE_ROLE) {
   emit Authorize(sender, data);
  }
}
```

FUEL3-10

CONTRACTS DON'T INHERIT FROM PAUSABLE

SEVERITY: Informational

PATH:

fuel-rollup/contracts/migrator/TokenMigrator.sol:L70-L76
fuel-rollup/contracts/sequencer/SequencerInterface.sol:L60-L66
fuel-rollup/contracts/vault/Vault.sol:L45-L51

REMEDIATION:

The TokenMigrator, SequencerInterface, and Vault contracts should inherit the pausable functionality from the Pausable abstract contract.

STATUS:

DESCRIPTION:

The **Pausable** abstract contract inherits from **PausableUpgradeable** and provides the implementation for the **pause()** and **unpause()** functions.

Nevertheless, the **TokenMigrator**, **SequencerInterface**, and **Vault** contracts have their own implementation of the **pause()** and **unpause()** functions without inheriting them from the **Pausable** contract.

```
// SPDX-License-Identifier: Apache-2.0
pragma solidity ^0.8.24;

import "../interfaces/ITokenMigrator.sol";
import "../interfaces/IERC20Mintable.sol";
import "../interfaces/ISequencerInterface.sol";
import "@openzeppelin/contracts-upgradeable/access/
AccessControlUpgradeable.sol";
import "@openzeppelin/contracts-upgradeable/utils/PausableUpgradeable.sol";

contract TokenMigrator is
 ITokenMigrator,
 AccessControlUpgradeable,
 PausableUpgradeable
{
 /// @custom:oz-upgrades-unsafe-allow state-variable-immutable
 address internal immutable V1_TOKEN;
 /// @custom:oz-upgrades-unsafe-allow state-variable-immutable
 address internal immutable V2_TOKEN;
 /// @custom:oz-upgrades-unsafe-allow state-variable-immutable
 uint256 internal immutable MIGRATION_RATIO;
 /// @custom:oz-upgrades-unsafe-allow state-variable-immutable
 address internal immutable LOCKUP;
 /// @custom:oz-upgrades-unsafe-allow state-variable-immutable
 uint256 internal immutable VESTING_PERIOD;

 bytes32 public constant PAUSER_ROLE = keccak256("PAUSER_ROLE");

 /// @notice Constructor disables initialization for the implementation
contract
 /// @custom:oz-upgrades-unsafe-allow constructor
 constructor(
  address v1Token,
  address v2Token,
  uint256 migrationRatio,
  address lockup,
  uint256 vestingPeriod
 ) {
  V1_TOKEN = v1Token;
  V2_TOKEN = v2Token;
```

```
    MIGRATION_RATIO = migrationRatio;
    LOCKUP = lockup;
    VESTING_PERIOD = vestingPeriod;
    _disableInitializers();
  }

  function initialize() external initializer {
    _grantRole(DEFAULT_ADMIN_ROLE, _msgSender());
    _grantRole(PAUSER_ROLE, _msgSender());
    __Pausable_init();
  }

  function migrate(
    uint256 amount,
    address validator
  ) external virtual override whenNotPaused {
    uint amountToMint = amount * MIGRATION_RATIO;

    // No need for safeTransfer - we know the tokens revert on failure
    IERC20(V1_TOKEN).transferFrom(_msgSender(), address(this), amount);

    // Migration forwards the tokens directly to the lockup & stake contract
    IERC20Mintable(V2_TOKEN).transfer(LOCKUP, amountToMint);
    ISequencerInterface(LOCKUP).handleMigration(
      _msgSender(),
      amountToMint,
      VESTING_PERIOD,
      validator
    );
  }

  function pause() external virtual onlyRole(PAUSER_ROLE) {
    _pause();
  }

  function unpause() external virtual onlyRole(PAUSER_ROLE) {
    _unpause();
  }
}
```


hexens
×
FUEL